Web Readability Radar

Technical documentation

Describes the local edition, version 1.0.0 (`server.mjs` SHA-256 `6f7230fb…`) · October 2026

Free online demonstration

A demonstration runs on the free plan of Cloudflare Workers so that the tool can be tried without installing anything: https://web-readability-radar.lecturabilidad-web.workers.dev/

What it can do

- Analyze a specific URL or a domain, in Spanish or English.
- Calculate INFLESZ/Szigriszt-Pazos for Spanish and Flesch Reading Ease for English.
- Extract and rank every fragment detected in the pages it manages to process, with reports and exports.
- Process at most 40 candidate pages per analysis and make up to 45 external web requests. It is therefore not suited to crawling an entire large domain.
- Read up to 1.5 million HTML characters per page, with at most 9 seconds per download.
- Keep no URLs, texts, results, or reports.

Main limits. It does not analyze local HTML files, does not run websites' JavaScript to reveal dynamic content, and stops exploring when it reaches the free budget. For exhaustive audits of hundreds or thousands of pages, the local edition described in this document has far greater capacity.

1. Overview

The essentials

- **What it is.** A bilingual (Spanish and English) application that locates the hardest-to-read pages and paragraphs of a website and records which pages were reviewed and why. It does not rewrite texts or certify accessibility.
- **How it runs.** A small Node.js server (the analysis engine and its API) and an interface that opens in the browser. In the local edition, both run on the user's own computer and the server only accepts connections from that computer (127.0.0.1). In a hosted deployment, the same server runs on the host and the interface reaches it at the host's address (section 3.6). There are no user accounts, no database, and no third-party dependencies.
- **What it does on the network.** It downloads the pages of the analyzed sites from the machine that runs the server and, if Chrome or Edge is installed there, can open them in a windowless (headless) browser to read the text that JavaScript generates.
- **Key point.** The current version is not ready to be exposed on the Internet as is: it has no authentication or per-user limits and does not filter destinations, so it could be used to send requests to an internal network. Publishing it requires the measures in section 6.

Technical summary

Aspect	Current situation
Language and runtime	JavaScript on Node.js 18 or later. The Windows package includes its own Node.js (24.19.0).

Aspect	Current situation
Dependencies	None from npm (no `npm install`). Two syllable tables in JSON (1.9 MB in total).
Server address	Local edition: `127.0.0.1:8787` (another free port if 8787 is busy); it does not accept connections from other computers. Hosted deployment: set by the hosting environment (section 3.6).
Calls from the interface	Relative paths (`/api/…`) to the same server that delivered the page. No IP address or host name is written into the interface.
Outbound network	HTTP/HTTPS to the analyzed sites, with the user agent `WebReadabilityRadar/1.0 (+local readability analysis)`. The interface loads no external resources and sends no data to the authors or to third parties.
Optional component	Chrome, Chromium, or Microsoft Edge installed on the machine that runs the server (or the path set in `CHROME_PATH`), for pages whose content is generated by JavaScript.
Storage	Nothing persistent. Results stay in memory until they are delivered to the browser; files are created only if the user downloads them.
Users and authentication	None.
Resource use	About 90 MB of memory when idle (measured with Node.js 22 on Linux). Each rendering launches a Chrome process; up to 8 can run at once.
Operating systems	Windows, macOS, and Linux.

2. What the tool does

The tool estimates how difficult a text is to read from the length of its sentences and words (classic readability indices; see Annex A). Its main contribution is not the index but **traceability**: it records how each page was found, whether it was analyzed, and why, so that someone else can check or repeat the review.

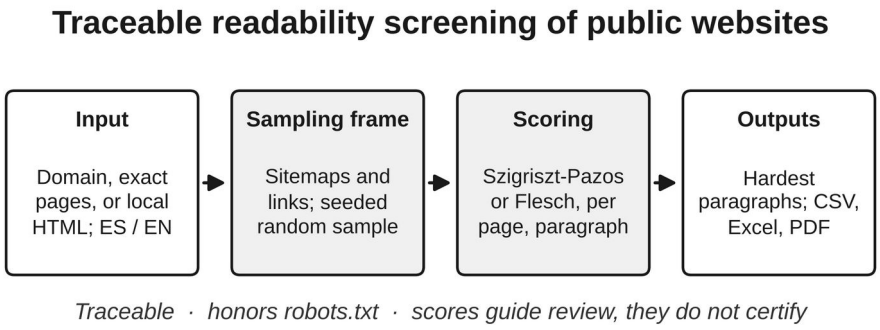


Figure 1. Overview: what goes in, how pages are selected, what is calculated, and what comes out.

Three input modes and two analysis modes

Input mode	What it analyzes	Example
Domain	Pages discovered from a starting address: a sample or all of them	Reviewing a faculty's website
One page	Only the addresses (URLs) entered	Checking a page before it is published
Local HTML	An .html file from the user's computer, without fetching anything	Reviewing a draft that is not yet online

Mode	What it does	Typical duration
Fast (default)	Bounded exploration; a random, reproducible sample of pages (12 by default); shows the 10 hardest paragraphs	Usually under a minute per domain
Complete	Analyzes every page it can discover (up to 10,000) and lists all paragraphs	Minutes to hours, depending on the size of the site

What it does not do. It does not simplify texts, does not check every Easy-to-Read guideline, and does not measure whether people understand the content. Its scores are signals for deciding what to review first.

3. How it is built

3.1 Components

Component	Files	Function	Where it runs
Interface	`public/index.html`, `app.js`, `styles.css`	Form, progress, result views, and downloads (CSV, Excel, PDF)	User's browser
Server and analysis engine	`server.mjs`	Serves the interface, provides the API, and performs discovery, retrieval, extraction, and scoring	Node.js process (user's computer or host)
Language resources	`resources/*.json`	Syllable counts: 4,962 Spanish and 126,045 English words	Loaded into memory at start-up
Renderer (optional)	Chrome, Chromium, or Edge on the same machine	Obtains text generated by JavaScript	Child process with a temporary profile
Launchers	`launch-windows.bat`, `launch-mac-linux.sh`, `runtime/node.exe`	Start the server and open the browser (local edition)	Operating system
Validation	`validation/`	Data and scripts used to validate the method	Not used at run time

3.2 Internal API

The browser and the server communicate through a simple HTTP API. The interface calls it with relative paths, so it always talks to the server that delivered it, wherever that server is hosted:

Path	Method	What it does
`/` and static files	GET	Serves the interface from the `public/` folder.
`/api/analyze`	POST (JSON, max. 8 MiB)	Starts an analysis. With `background: true` it immediately returns a job identifier (`jobId`); otherwise it waits and returns the full result.
`/api/analyze-status?jobId=…`	GET	Returns the job state (`running`, `complete`, or `error`), its progress, and, when finished, the result.
`/demo/es`, `/demo/en`	GET	Built-in test pages.

The interface polls the job state every 0.75 seconds. Each job identifier is a random 128-bit value. A job is removed from memory as soon as its result is delivered or, at the latest, one minute after it finishes; in addition, starting a new job removes any job older than 15 minutes. The result is a single JSON object containing the summary, the criteria applied, the record of every candidate URL, the pages, and the paragraphs.

3.3 Data handled and retention

Data	Where	For how long
URLs entered and parameters	Server and browser memory	While the analysis runs
Downloaded HTML and extracted text	Server memory	While each page is processed
Results (pages, paragraphs, URL record)	Server memory, then browser memory	Server: until delivered (at most 1 minute after finishing). Browser: until the tab is closed or another analysis starts
Temporary Chrome profile	System temporary folder of the server machine	Deleted after each rendering
Exported files (CSV, Excel, PDF)	User's computer	Only if the user downloads them
Logs	Server console	Start-up messages only; requests are not logged

Analyzed pages may contain published personal data (staff names, e-mail addresses, or telephone numbers), which will then appear in exported paragraphs. The tool displays a privacy notice when it opens.

3.4 Network behaviour

Direction	From → to	When	Notes
Inbound	Browser → server (local edition: `127.0.0.1:8787`)	Always	Local edition: only from the same computer. No authentication in any edition
Outbound	Server → robots.txt, sitemaps, and pages of the domain	Domain mode, during discovery	Follows redirects; at most 12 s per request
Outbound	Server → selected pages	During the analysis	Up to 8 at a time; no pause between requests

Direction	From → to	When	Notes
Outbound	Headless Chrome → the page and all its resources (scripts, images, third-party services)	Exact pages and pages that seem to be generated by JavaScript	Chrome runs the JavaScript of the analyzed site

All outbound requests leave from the IP address of the machine that runs the server: the user's computer in the local edition, or the host in a hosted deployment.

3.5 How to try it (local edition)

1. **Windows:** unzip the package and run `launch-windows.bat`. It includes Node.js; no installation or administrator rights are needed.
2. **macOS or Linux:** with Node.js 18 or later installed, run `sh launch-mac-linux.sh` from the package folder.
3. **Optional:** install Chrome, Chromium, or Edge (or set `CHROME_PATH`) to analyze dynamic pages.
4. The interface opens at `http://127.0.0.1:8787/`. For a quick test, analyze `http://127.0.0.1:8787/demo/en` in "One page" mode. To stop the tool, press Ctrl+C in the terminal window.

3.6 Server address and hosting

The address of the server is the only element that depends on where the tool is finally hosted. The current code handles it as follows:

Element	Current behaviour (version 1.0.0)	Implication for hosting
Calls from the interface	Relative paths (`/api/analyze`, `/api/analyze-status`); the example button uses the current origin (`location.origin`)	No change needed: the interface follows the server's address
Listening address	Fixed in the code to `127.0.0.1` (`server.listen(port, '127.0.0.1')` in `server.mjs`)	Valid behind a reverse proxy on the same machine. In a container, or if the proxy runs on another machine, the address must become a setting (for example, a `HOST` environment variable)
Listening port	`PORT` environment variable or `--port` argument; default 8787. If the port is busy, the server silently switches to a random free port	Set the port explicitly. A silent change of port would break the proxy configuration, so failing on a busy port is preferable
Address shown at start-up	The console prints, and the launchers open, `http://127.0.0.1:<port>/`	Informational only; the interface does not use it
Edition detection	The interface switches to the online mode (a single bounded response, no background jobs) when its host name ends in `workers.dev`	Affects only the Cloudflare demonstration. A full deployment under a university domain runs in normal mode; a limited edition hosted elsewhere would need an explicit setting instead of this check
Outbound requests	Leave from the IP address of the machine that runs the server (in the Cloudflare demonstration, from Cloudflare's network)	Analyzed sites will see, and may log or rate-limit, the host's IP address; the user agent identifies the tool

Editions. This document describes the local edition, which is the reference implementation. A demonstration edition runs on the free plan of Cloudflare Workers so that the tool can be tried without installing anything. Its capabilities and limits are summarized at the start of this document. It is a preview, not the intended production hosting: its server code is a separate implementation not covered by this document, and the pages it analyzes are requested from Cloudflare's network.

4. What happens during an analysis

Each analysis goes through six stages. Only stages 2 and 3 contact external websites.

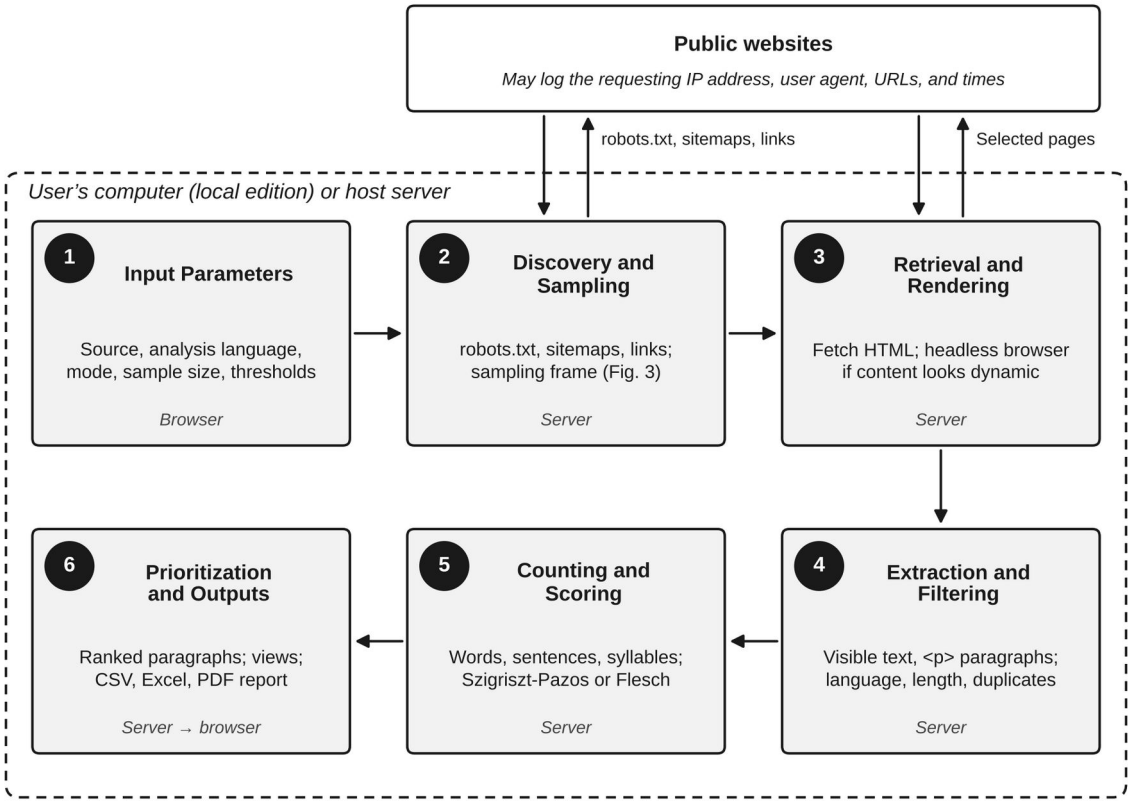


Figure 2. The six stages of an analysis and where each one runs. The dashed line encloses what runs on the user's computer (local edition) or on the host server (hosted deployment).

Stage	What it does	Contacts the Internet?
1. Input parameters	The user chooses the input mode, the analysis language (which sets the formula), the mode, the sample size, and the minimum word thresholds	No
2. Discovery and sampling	Reads robots.txt and the sitemaps, follows internal links, builds the list of candidate pages, and chooses which ones to analyze (Figure 3)	Yes, in domain mode only
3. Retrieval and rendering	Downloads each page; if it seems to be built with JavaScript, opens it in headless Chrome and keeps the version with more text	Yes
4. Extraction and filtering	Separates the visible text and the paragraphs (<p>); discards menus, footers, sidebars, forms, and dialogs; excludes text in another language, too short, or duplicated	No

Stage	What it does	Contacts the Internet?
5. Counting and scoring	Counts words, sentences, and syllables and calculates the index of each page and paragraph (Annex A)	No
6. Prioritization and outputs	Sorts paragraphs from hardest to easiest and prepares the views and downloads	No

4.1 How pages are chosen

In domain mode, the tool reads the site's robots.txt and does not download the addresses it disallows; it reads the sitemaps (up to 50) and follows internal links level by level. Every address found is recorded with its origin, its section (the first segment of its path), its depth, and, where applicable, the reason for its exclusion. In fast mode, the sample is drawn at random and spread across sections from a **seed** that is recorded: the same seed and the same list of candidates always produce the same sample. In complete mode, all permitted candidates are analyzed. In "One page" and "Local HTML" modes, no other pages are sought and robots.txt is not consulted, because the user has chosen the content.

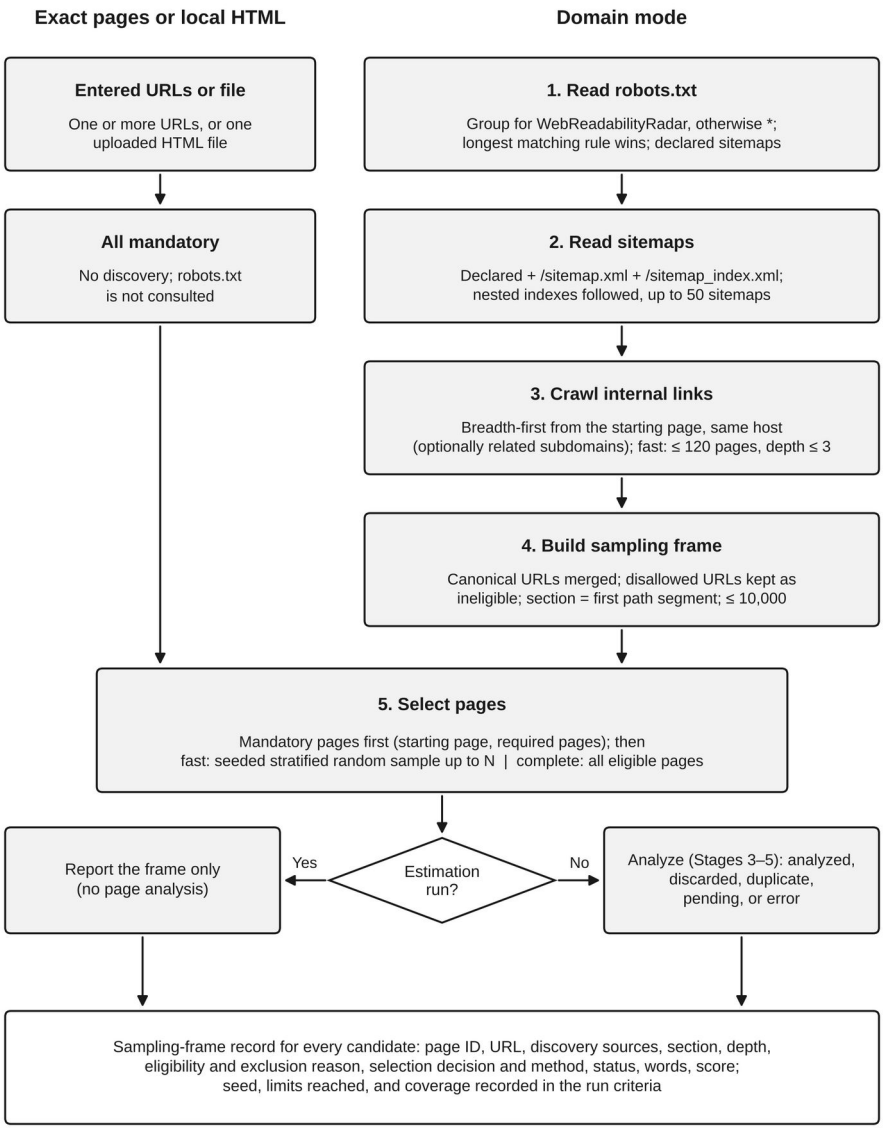


Figure 3. Discovery and selection of pages. Every candidate URL is recorded, even if it is not analyzed.

4.2 Page statuses

Status	Meaning	Included in results?
Analyzed	Downloaded, enough text extracted, and indices calculated	Yes
Discarded	Not enough text, or text in a language other than the one selected	No; the reason is shown
Duplicate	Its text is identical to that of another page already included	No; the original page is indicated
Pending	Not processed in time in fast mode	No; coverage is marked as partial
Error	Network failure, timeout, non-HTML file, or excessive size	No; the message is shown

5. Operating limits

These limits are set in the code and ensure that every analysis finishes within a bounded time:

Control	Value	What happens when it is reached
Candidate URLs and pages per analysis	10,000	Discovery stops and partial coverage is reported
Discovery in fast mode	120 crawled pages, 3 levels, and 20 s per domain	Partial coverage reported in the result
Discovery in complete mode	No level limit; 120 s per domain	Partial coverage reported in the result
Sitemaps read	50 per domain	Partial coverage reported in the result
Analysis time in fast mode	30 s per analysis	Unprocessed pages are marked “pending”
Time per request	12 s (up to 20 s with rendering)	The page is marked “error”
Pages in parallel	8	The others wait their turn
Size of a page or file	6,000,000 characters	Rejected as a whole; never analyzed partially
Size of an API request	8 MiB	Error 413
Simultaneous jobs	No limit	Risk of exhausting resources if the service is published (section 6)

Interpreting coverage. The fast-mode sample is random with respect to the pages discovered, not to the whole website, and it spreads pages equally across sections, so small sections are relatively overrepresented. The result always states which limits were reached.

6. Security, privacy, and governance requirements

Recommended requirements for any published version:

Area	Requirement	Priority
Requests to internal destinations (SSRF)	Allow only http and https to public addresses; check this after DNS resolution and at every redirect; block localhost, private networks, link-local addresses, and metadata services	Before publishing
Server address and port	Make the listening address and port explicit settings, expose the server only through the HTTPS reverse proxy, and replace the host-name-based edition check (section 3.6)	Before publishing
Access and quotas	University sign-in or a list of authorized users; per-user limits on jobs, pages, and time	Before publishing
Resources	Job queue with maximum concurrency, maximum time per job, cancellation, and CPU and memory limits per container	Before publishing
Headless browser	Dedicated container, unprivileged user, Chrome sandbox enabled, no access to the internal network or to server files	Before publishing (if rendering is kept)
Respect for analyzed sites	Keep robots.txt compliance (already implemented in domain mode); limit the request rate per site; include a contact URL in the user agent	Before publishing
Data protection (GDPR)	Define what is stored, for how long, and who has access; minimal logs; review with the Data Protection Officer	Before publishing
Exports	Neutralize formulas in CSV and Excel files; review HTML escaping in reports	Before publishing
Transport and headers	HTTPS, HSTS, a Content Security Policy, and validation of the Host and Origin headers	Before publishing
Maintenance	Identified versions, a changelog, updates of Node.js and Chrome, and automated tests that warn if the results of the validated algorithms change	Recommended
Monitoring	Error logging and usage metrics without personal data	Recommended

Annex A. How readability is calculated

The user chooses the language of the content at the start; this choice sets the formula, the categories, and the language filter, independently of the interface language.

Language	Formula
Spanish (Szigriszt-Pazos, INFLESZ scale)	P = 206.835 − 62.3 × (syllables / words) − (words / sentences)
English (Flesch Reading Ease)	RE = 206.835 − 1.015 × (words / sentences) − 84.6 × (syllables / words)

- **Scores:** the higher, the easier. Scores are capped at 100; negative values are kept because they indicate very difficult text.

- **Page and paragraph:** the index of a page is calculated from the counts of all its visible text, not as the mean of its paragraphs. Each eligible paragraph also receives its own index.
- **Sentences:** in Spanish, they are split at periods, semicolons, colons, question marks, exclamation marks, and em dashes, after web addresses and decimal numbers have been protected. In English, at periods, question marks, and exclamation marks.
- **Syllables:** the built-in tables are consulted first (4,962 Spanish words from pylabeador; 126,045 English words from the CMU Pronouncing Dictionary); words not in the tables are counted with orthographic rules.

Categories and colors

Spanish (INFLESZ)	English (Flesch)	Color in the interface
80 or more: very easy	80 or more: very easy / easy	Green
65–79.9: fairly easy	70–79.9: fairly easy	Light green
55–64.9: normal	60–69.9: plain English (standard)	Yellow
40–54.9: somewhat difficult	50–59.9: fairly difficult	Orange
Below 40: very difficult	Below 50: difficult / very difficult	Red

In the validation, scores agreed with those of the INFLESZ program (mean difference of 1.0 point on 100 texts) and with those of Microsoft Word (0.8 points on 100 texts, on Word's 0–100 scale).

Annex B. Glossary

Term	Meaning
Readability index	A number that estimates how difficult a text is from the length of its sentences and words
Paragraph (fragment)	Each complete <p> element of a page; the unit that is ranked for review. The interface calls them "fragments"
Candidate URL	An address found during discovery, whether or not it is analyzed
Sampling frame	The record of all candidate URLs with their origin, eligibility, selection, and result
Seed	A value that determines the random sample; the same seed and the same candidates produce the same sample
Fast / complete mode	Bounded analysis of a sample / analysis of every permitted page that is discovered
Headless rendering	Opening a page in a browser without a window to obtain the text generated by JavaScript
robots.txt	A file in which a site states which parts automated programs should not crawl
Sitemap	An XML file in which a site lists its pages
SSRF	Server-Side Request Forgery: using a server to send requests to internal destinations it should not reach

Term	Meaning
Reverse proxy	An intermediate server that receives public requests, applies security controls, and forwards them to the internal service